RICAM Workshop · 2-6 November 2026

```
0 01111 0000000000
0 10000 1000000000
1 01101 0101010101
0 00000 0000000001
0 11111 0000000000
```

Inside the Black Box

Bit-accurate models of NVIDIA Tensor Cores and AMD Matrix Cores

Faizan A. Khattak
Joint work with Mantas Mikaitis
School of Computer Science, University of Leeds

Mixed-precision matrix multipliers

Floating-point **matrix multiply-accumulate (MMA)** is a hardware operation introduced in GPUs relatively recently.

$$\underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}}_{\text{binary16 or binary32}} = \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}}_{\text{binary16 or binary32}} + \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}}_{\text{8/16/19-bit FP}} \times \underbrace{\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix}}_{\text{8/16/19-bit FP}}$$

$$D = C + A \times B,$$

Hardware matrix multipliers in mixed precision

- MMA is ubiquitous and drives algorithm research (mixed precision).
- Example is 4×4, but dimensions differ across architectures.

MOTIVATION

Matrix multipliers now power far more than AI

01

Built for AI

Matrix multiplication drives both training and inference. GPUs accelerate it in dedicated units: tensor cores on NVIDIA, matrix cores on AMD, XMX cores on Intel GPU (AMX cores on Xeon CPUs)

02

Scientific Computing

Their speed draws in scientific computing: mixed-precision iterative refinement, emulation of higher precision GEMMs, and more.

03

Low-precision inputs

4-, 6-, 8-, 16- and 19-bit floating-point inputs, with wider accumulation inside each inner product.

IEEE 754: What Is (and Isn't) Standardised?

Standardised

- Correctly rounded $+$, $-$, $\times$, $\div$, $\sqrt{\ }$ and fused multiply-add ($a \times b + c$).
- Elementary functions (exp, log, sin, cos, pow) are only *recommended* to be correctly rounded; most libraries do not guarantee it.

Not standardised: order of operations

- Multiplication is commutative, but associativity and distributivity fail in general.
- The order of summing n terms is not fixed, so result vary with implementation.

Matrix multiplication

- Clause 9.4 (IEEE 754-2019): reductions may associate in any order or use a wider format, therefore, implementations may vary.
- Hardware behaviour is often undocumented and changes between generations, which hurts reproducibility and portability and makes errors hard to explain.

Goals of the Project

- Determine the numerical features of tensor cores on 5 generations of NVIDIA GPUs

GPU Model	Architecture	SM Compute Capability
V100	Volta	70
A100/A30	Ampere	80
A2/A40	Ampere	86
L40S/RTX-1000	Ada Lovelace	89
H100	Hopper	90
B200/RTX Blackwell	Blackwell	100/120

Goals of the Project (cont.)

- Determine the numerical features of tensor cores on 5 generations of NVIDIA GPUs
- Determine the numerical features of matrix cores on 3 generations of AMD GPUs

GPU Model	Architecture
MI100	CDNA 1
MI210, MI250	CDNA 2
MI300A, MI300X	CDNA 3

- Develop MATLAB models for mixed-precision experimentation.

EXAMPLE ILLUSTRATION

MATLAB Tensor Core v0.6

```
inopts.format  = 'fp8-e4m3';
outopts.format = 'binary32';
A = cpfloat(randn(4,32), inopts);
B = cpfloat(randn(32,4), inopts);
C = cpfloat(randn(4,4), outopts);
alpha = 1; beta = 1;

B200TC(alpha, A, B, beta, C, ...
       inopts.format, outopts.format)
H100TC(alpha, A, B, beta, C, ...
       inopts.format, outopts.format)
```

B200

```
 11.2245   -0.9896    2.1935   -0.4385
-10.0326    4.4496    4.8246    2.4309
 -2.4434    5.0158    2.6123    2.6869
  4.4591   -9.3572  -11.5814   -0.7091
```

H100

```
 11.2236   -0.9897    2.1934   -0.4385
-10.0322    4.4492    4.8247    2.4312
 -2.4434    5.0156    2.6125    2.6868
  4.4585   -9.3574  -11.5811   -0.7092
```

Same inputs, but slightly different results, which accumulate over repeated multiplications.

High-precision matrix multiply Example

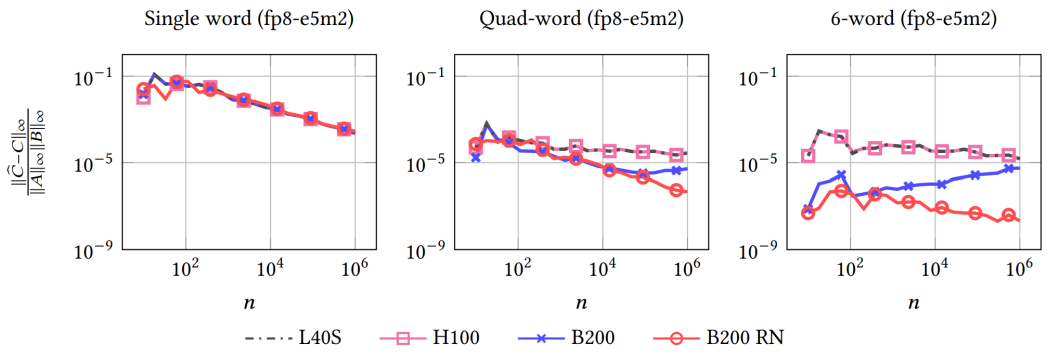
- Use low-precision matrix multiplication to emulate high precision [Mary and Mikaitis, 2025].
- Split the inputs into a p-word representation:

$$A_i = \mathrm{fl}_{\mathrm{low}}\Big(A - \sum_{k=1}^{i-1} A_k\Big), \qquad B_j = \mathrm{fl}_{\mathrm{low}}\Big(B - \sum_{k=1}^{j-1} B_k\Big),$$

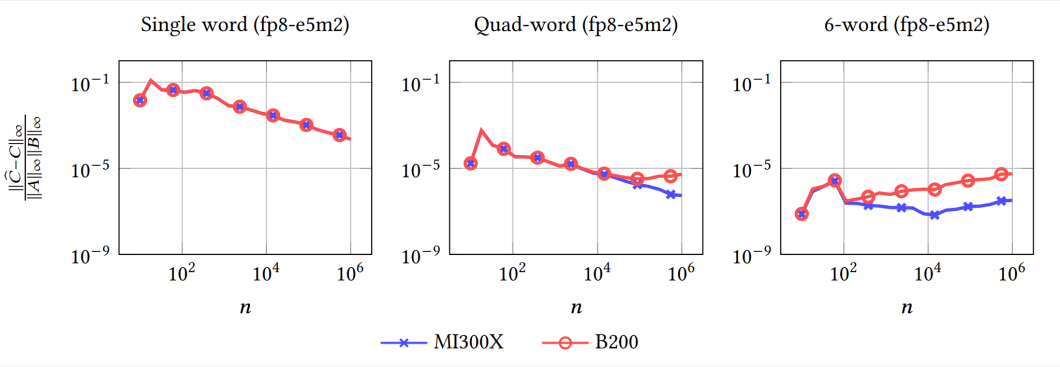
  (scaling is omitted for simplicity).
- The product is obtained as

$$C \approx \sum_{i=1}^{p}\sum_{j=1}^{p} A_i B_j.$$

High-precision matrix multiply Example (contd.)



High-precision matrix multiply Example (contd.)



How do we determine Model

We focus on a single element of output matrix D : $d = \sum_{\ell=1}^{k} a_\ell b_\ell + c$, and determine following (but not limited to)

1. how products are computed, its precision, and if any possible rounding
2. architecture type: multi-term alignment, or sequential FMA
3. if multi-term alignment: what its block size N_{FMA}, extra aligment bits beyond output precision n_{eab}, rounding at alignment, addition of accumulation term c (late or early), any intermediate normalisation
4. subnormal input and output support, overflow and underflow, and special inputs

through **generalized numerical feature testing** (GNFT):

GNFT: Generalised Numerical Features Testing

- Each feature test is built around an input test vector $X = \{a, b, c\}$.
- For each test, define candidate outputs $Y_1, Y_2, \ldots$ that cover the possible behaviours of the MMA design.
- Each test $\{X, Y_1, Y_2, \ldots\}$ is written in terms of p_{in}, p_{out} (input and output precision) and other parameters, not fixed constants.
- The tests therefore apply to current and future floating-point input formats.
- Procedure: feed X to the matrix multiplier and compare its output d:

$$d = Y_1 \;\Rightarrow\; \text{feature present}, \qquad d = Y_2 \;\Rightarrow\; \text{feature absent}, \qquad \ldots$$

- GNFT gives the initial approximation of the model (Step 1 of the refinement algorithm).

ISSM: Input Space Search Method

The input space of dot products is huge: how do we find inputs that expose mismatches between the GPU and the GNFT-approximated model?

- **Strategy 1**: N_{ens} samples for $a, b \in \mathbb{R}^K$ and c drawn normal distribution.
- **Strategy 2**: Independent Exponent–Mantissa Sampling

$$a_\ell = s_\ell 2^{e_\ell}, \quad e_\ell \overset{\text{i.i.d.}}{\sim} \mathcal{U}(e_a^-, e_a^+), \quad s_\ell \overset{\text{i.i.d.}}{\sim} \mathcal{U}(-2 + 2^{-p_{\text{in}}+1},\, 2 - 2^{-p_{\text{in}}+1}), \quad \ell = 1, \ldots, k$$

$$b_\ell = s_\ell 2^{e_\ell}, \quad e_\ell \overset{\text{i.i.d.}}{\sim} \mathcal{U}(e_b^-, e_b^+), \quad s_\ell \overset{\text{i.i.d.}}{\sim} \mathcal{U}(-2 + 2^{-p_{\text{in}}+1},\, 2 - 2^{-p_{\text{in}}+1}), \quad \ell = 1, \ldots, k$$

$$c = s 2^{e}, \quad e \overset{\text{i.i.d.}}{\sim} \mathcal{U}(e_c^-, e_c^+), \quad s \overset{\text{i.i.d.}}{\sim} \mathcal{U}(-2 + 2^{-23},\, 2 - 2^{-23})$$

- **Strategy 3**: Each bit independently sampled from a Bernoulli distribution with equal probability.

We construct an ensemble $\mathcal{S}$ containings vectors from all three strategies

With $m = 0$ mismatches in N_{ens} trials, Bayesian upper bound on mismatch is $\ln(1/\varepsilon)/(N_{\text{ens}} + 1)$. For $N_{\text{ens}} = 10^7$ and $\varepsilon = 10^{-6}$ that is below 1.4×10^{-6}.

Approximation and Refinement of Models

Machine step: Apply GNFT using generalised test vectors.
Engineer step: Create an initial approximation of the model.
while true **do**
    Machine step: Generate an ISSM ensemble $\mathcal{S}$.
    Machine step: Compare GPU and the approx. model over $\mathcal{S}$.
    if mismatches are detected between GPU and model outputs **then**
        Engineer step: Inspect failure cases and refine the model.
        Engineer step: Modify ISSM to include similar cases to the failure cases.
        Engineer step: Update GNFT for detecting any new features.
    else
        Engineer interpretation: The model is considered sufficiently accurate.
        break
    end if
end while

Tested Input Precision Formats

NVIDIA tensor cores

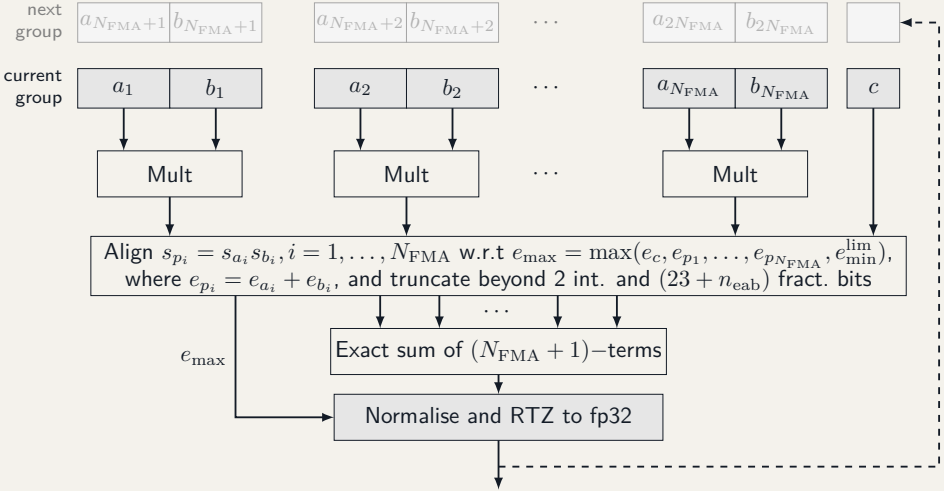
Arch.	GPUs	Inputs
Volta	V100	fp16
Ampere	A100, A2, A30	fp16, bf16, tf32
Ada	L40S/RTX-1000	fp8, fp16, bf16, tf32
Hopper	H100/GH200, H200	fp8, fp16, bf16, tf32
Blackwell	B200/RTX PRO 6000	fp8, fp16, bf16, tf32

AMD matrix cores

Arch.	GPUs	Inputs
CDNA 1	MI100	fp32, fp16, bf16
CDNA 2	MI210, MI250	fp32, fp16, bf16
CDNA 3	MI300A, MI300X	fp8, fp16, bf16, tf32, fp32

fp8 covers both E4M3 and E5M2. Outputs are in fp32, while only NVIDIA support fp16 output in fp16/fp8 input formats.

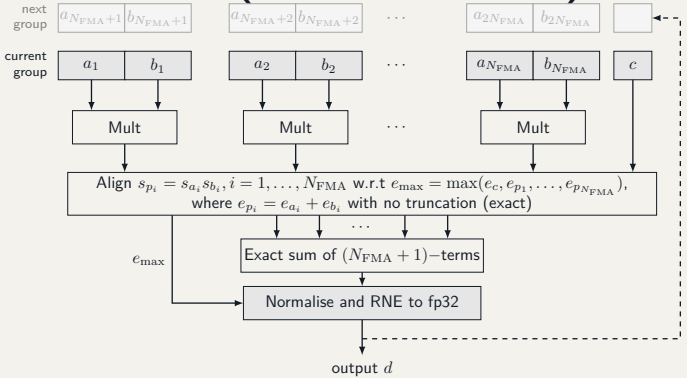
NVIDIA Tensor Core Model



NVIDIA TC Features

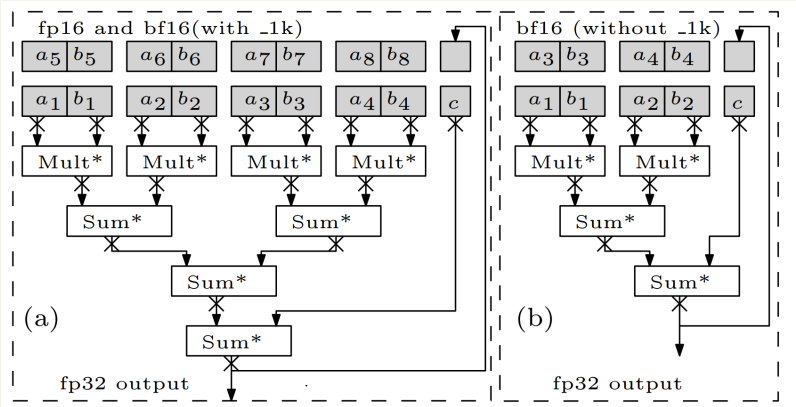
In	Out	GPUs	n_{eab}	N_{FMA}	$e_{\min}^{\lim}$	Final Round.
fp8	fp32	B200, RTX PRO	2	32	–	Trunc
		H100, H200	-10	32	–	Trunc
		L40S, Ada RTX 1000	-10	16	–	Trunc
fp8	fp16	B200, RTX PRO	2	32	-21	RNE
		H100, H200	-10	32	-21	RNE
		L40S, Ada RTX 1000	-10	16	-20	RNE
fp16/bf16	fp32	H100/H200/B200/RTX PRO	2	16	-133	Trunc
		L40S, Ada RTX 1000	1	8	-132	Trunc
		A100, A2, A30	1	8	-132	Trunc
		V100	0	4	-19	Trunc
tf19	fp32	H100/H200/B200/RTX PRO	2	8	-133	Trunc
		L40S, Ada RTX 1000	1	4	-132	Trunc
		A100, A2, A30	1	4	-132	Trunc
fp64	fp64	H100/H200/B200/RTX PRO	–	1	–	all 4
		A100	–	1	–	all 4

AMD Matrix Cores (CDNA 1 Arch.)



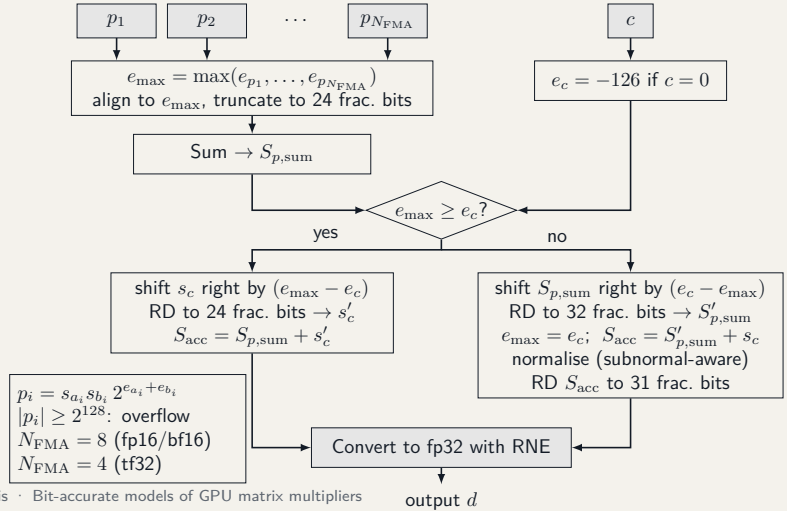
- $N_{\text{FMA}} = 4$ and 2 for fp16 and bf16 formats, respectively
- Correctly rounded architecture (Kulisch accumulator, i.e., $n_{\text{eab}} = \infty$)
- fp32 input format implements sequential FMA

AMD Matrix Cores (CDNA 2 Arch.)



Mult*: Products are in fp32 precision format; ×: subnormal flushed to zero; Sum*: Sum plus RNE

AMD Matrix Cores (CDNA 3 Arch.)



SOFTWARE

MATLAB Tensor Core, open source

```
% D = alpha*A*B + beta*C, as the GPU computes it
D = A100TC(alpha, A, B, beta, C, in_format, out_f
D = B200TC(alpha, A, B, beta, C, in_format, out_f
D = V100TC(alpha, A, B, beta, C, out_format);

% Custom designs: Generic_BFMA_TC, set e.g.
% neab (extra alignment bits), fma (block size),
% frmode ('rne','rz','rd','ru'), inter_pattern
```

GPU models
NVIDIA Volta to Blacwell and AMD CDNA 1, 2 and 3.

Generic block-FMA model
Instantiate custom tensor-core variants feature by feature.

Compatibility
MATLAB or GNU Octave; callable from Python.

github.com/north-numerical-computing/MATLAB-tensor-core

Use Cases of Bit-Accurate Tensor Core Models

Algorithm development

Run hardware-exact experiments without the GPU; tune mixed-precision and multi-word algorithms for a specific device.

Error analysis

Derive and check rounding-error bounds using the true accumulator width, block size and rounding mode.

Reproducibility

Explain why the same code gives different results on different GPUs; predict numerical changes when porting.

Testing

Validate test vectors before running them on real hardware; start characterising new GPU generations.

Feature isolation

Change one feature at a time, e.g. B200 with round-to-nearest, and measure its effect alone.

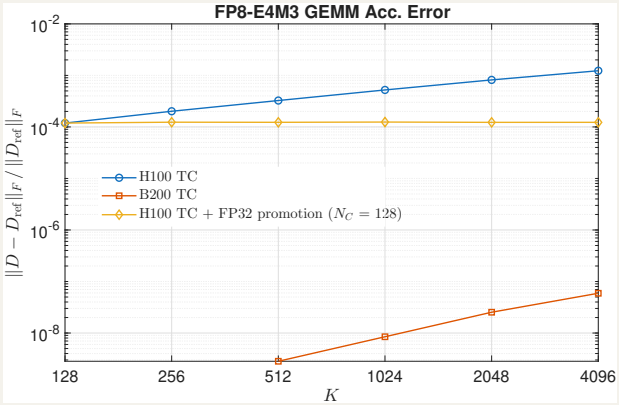
Hardware design

Explore new alignment widths, block sizes, rounding modes and low-precision formats before building them.

Cross-hardware emulation

Known features enable *bit-exact emulation* of one tensor core on other hardware, e.g. B200 tensor-core arithmetic on Volta CUDA cores. It is slower, but useful for reproducibility, debugging and validation.

Reproducing DeepSeek-V3's FP8 Observation



$A \in \mathbb{R}^{64\times K}$, $B \in \mathbb{R}^{K\times 64}$, fp8-e4m3 inputs, fp32 output.

- DeepSeek-V3: FP8 GEMMs on H800 (Hopper) keep only ≈14 bits; fix by promoting partial sums to FP32 every $N_C = 128$.
- **H100 model:** error grows with K (13 fraction bits, truncation).
- **Promotion:** error stays at its $K = 128$ level.
- **B200:** $\sim 10^4\times$ more accurate (25 fraction bits); Blackwell fixes it in hardware.

THANK YOU

Questions?

NVIDIA paper (2026)

AMD paper (2026)

Accurate Models of NVIDIA Tensor Cores · ACM TACO, doi:10.1145/3830409

Accurate Models of AMD Matrix Cores · arXiv:2609.14845

Generalized Methodology for Determining Numerical Features of Hardware Floating-Point Matrix Multipliers: Part I · arXiv:2609.14845

Toolbox · github.com/north-numerical-computing/MATLAB-tensor-core

Faizan A. Khattak and Mantas Mikaitis · {f.a.khattak,m.mikaitis}@leeds.ac.uk